

iterator / begin / end 정리

STL 컨테이너의 원소를 순서대로 가리키고 이동하는 표준 접근 방식입니다.

사용처 (Where)

- 컨테이너 종류와 상관없이 순회할 때
- STL 알고리즘에 범위를 전달할 때
- erase, insert 위치를 지정할 때

방법 (How to Use)

- begin()은 첫 원소 위치
- end()는 마지막 다음 위치
- *it로 값 접근
- ++it로 다음 원소 이동

기본 정보

- 필요 헤더: 컨테이너 헤더에 포함
- 기본 선언: auto it = v.begin();

왜 써야 할까? (Why)

- STL 알고리즘의 공통 언어 역할
- 컨테이너가 달라도 비슷한 방식으로 순회 가능
- erase/insert 위치 표현에 필요

주요 함수

함수/문법	의미
begin()	첫 원소 반복자
end()	마지막 다음 반복자
rbegin()/rend()	역방향 순회
*it	반복자가 가리키는 값
erase(it)	해당 위치 삭제

기본 코드 예시

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30};

    for (auto it = v.begin(); it != v.end(); ++it) {
        cout << *it << ' ';
    }
}
```

iterator / begin / end 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
반복자 이동	컨테이너마다 다름
vector ++it	O(1)
list ++it	O(1)
거리 계산	vector는 빠르고 list는 선형

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
표현력	의도를 타입으로 직접 표현	구조체/클래스를 직접 작성	배열/struct 수동 관리
코드 길이	짧고 표준적인 형태	필드와 생성자 작성 필요	초기화와 접근 실수 가능
유지보수	다른 STL과 자연스럽게 연동	설계가 늘어날수록 관리 필요	가독성이 떨어지기 쉬움
추천 상황	간단한 값 묶음/조건 전달	도메인 의미가 큰 객체	C API와 맞춰야 할 때

응용: 조건에 맞는 원소 삭제

erase가 반환하는 다음 반복자를 받아야 안전하게 순회를 계속할 수 있습니다.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5};

    for (auto it = v.begin(); it != v.end(); ) {
        if (*it % 2 == 0) it = v.erase(it);
        else ++it;
    }

    for (int x : v) cout << x << ' ';
}
```

처음 배울 때 자주 하는 실수

- end()는 실제 원소가 아니므로 역참조하면 안 됨
- vector에서 삽입/삭제 후 기존 iterator가 무효화될 수 있음
- 범위 기반 for가 더 쉬운 경우에는 그것을 사용

비슷한 항목과 차이

- 포인터는 배열 중심, iterator는 STL 컨테이너 전체에 사용
- range-based for는 단순 순회에 적합
- index는 vector/string처럼 인덱스 가능한 자료에 적합

한 줄 정리: `iterator / begin / end`은(는) '컨테이너 종류와 상관없이 순회할 때' 상황에서 먼저 떠올리면 좋은 STL입니다.